

Evaluación de la robustez de modelos de detección de *spoofing* en audio mediante aprendizaje por refuerzo adversarial

Autor:
Esp. Prof. Alejandro Lloveras

Este documento fue realizado en el curso de Aprendizaje por Refuerzo I
de la Maestría en Inteligencia Artificial, el 20 de abril de 2026.

Descripción del Problema

Introducción general al tema

La democratización de los modelos generativos multimodales y los recientes avances técnicos que permiten sintetizar audio o video a partir de un simple texto (*prompt*), así como la incorporación de modelos especializados en la transformación de archivos en estos formatos, abren un nuevo vector de ataque a sistemas de identificación del hablante y facilitan la realización de estafas basadas en la suplantación de identidad (*deepfake*).

Por esta razón, se vuelve imprescindible contar con herramientas de ciberseguridad para el análisis de audio que sean capaces de identificar voces sintetizadas, así como detectar indicios de manipulación, para alertar al usuario antes de que se concrete el ataque, lograr sistemas automáticos más robustos o analizar evidencia forense en contextos jurídicos.

El estado del arte en la detección de audio *spoofing* ha evolucionado significativamente desde el uso de descriptores artesanales y clasificadores estadísticos hacia arquitecturas de aprendizaje profundo *end-to-end* que operan directamente sobre el dominio del tiempo o representaciones espectro-temporales de alta resolución.

Los avances más recientes dependen de Redes de Grafos (GNN), que permiten capturar artefactos de síntesis (TTS) y conversión de voz (VC) con mayor precisión al tratar los componentes espectrales y temporales de manera unificada, superando las limitaciones de las convoluciones estándar para detectar anomalías globales.

Como resultado de esta línea de investigación en grafos y atención integrada, se presenta AASIST3, una arquitectura que intenta maximizar la robustez frente a ataques no vistos y optimizar la eficiencia computacional en la discriminación de audio sintético.

Descripción General del Proyecto

Los detectores de vanguardia actuales, como AASIST3, dependen de Redes de Atención en Grafos (GAT), para el modelado temporal/espacial, y de las Redes de Kolmogorov-Arnold (KAN), para superar las limitaciones de los perceptrones multicapa (MLP) tradicionales en la modelación de relaciones no lineales complejas. Si bien las KAN ofrecen una aproximación de funciones superior, pueden introducir vulnerabilidades adversariales únicas debido a su dependencia de funciones de activación aprendibles (*splines*) en las aristas, en lugar de activaciones fijas en los nodos.

El presente proyecto propone aprovechar las estructuras matemáticas específicas de las Redes de Kolmogorov-Arnold para manipular el audio sintético con el fin de explotar los límites de decisión de las arquitecturas neuronales basadas en *splines*.

KON-Artist: Kolmogorov-Arnold Obliterated Network

KON-Artist (Repositorio GitHub) es un sistema de Aprendizaje por Refuerzo Adversarial (ARL) diseñado para realizar pruebas de estrés y vulnerar el modelo de detección de spoofing de audio del SOTA. Se trata a AASIST3 como un **Crítico** "congelado" (*frozen*) dentro de un entorno de *Reinforcement Learning* (RL). Se entrena un **Agente** (el generador) que aprende a ajustar los parámetros de un ***pipeline* de procesamiento de audio** (DSP) que manipula los audios maliciosos (clase "*spoofed"*) para maximizar la probabilidad de que sean clasificados por el crítico como legítimos (clase *"bonafide"*).

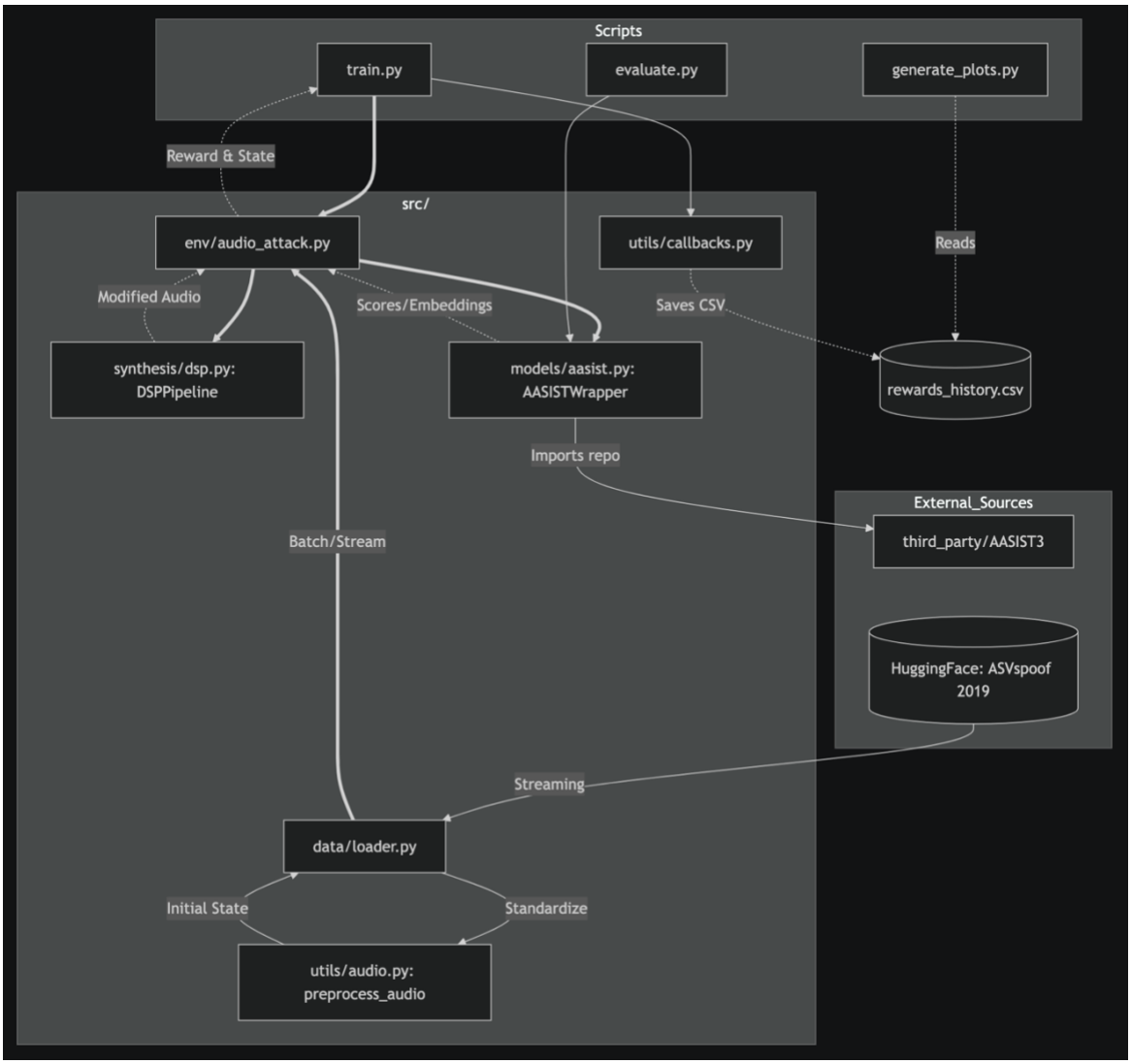


Fig.1 – Diagrama del sistema.

La arquitectura **AASIST3** fue clonada del repositorio oficial y los pesos del modelo entrenado, obtenidos desde Hugging Face (MTUCI/AASIST3).

El dataset utilizado fue **ASVspoof 2019,** mismos datos con que fue entrenado AASIST3, que funciona como *benchmark* en este dominio. Son datos públicos, accesibles desde el

sitio oficial de la competencia, Kaggle o Hugging Face. Para la implementación se utilizó la versión de Hugging Face. Se decidió entrenar el agente con los mismos datos de entrenamiento utilizados por el crítico (AASIST3), filtrando sólo los audios *spoofed*. Esto complejiza la tarea del generador, ya que debe lograr confundir al modelo aún con datos que ya ha visto.

Durante el desarrollo se realizaron las pruebas necesarias para el desarrollo modular subsiguiente:

- **01_AASIST_basic.ipynb:** Implementación e inferencia básica del modelo preentrenado AASIST3 para la clasificación de audio genuino frente a ataques sintéticos.
- **02_AASIST_evaluation.ipynb:** Evaluación del desempeño de AASIST3 mediante el cálculo de métricas de error (EER) sobre protocolos estándar de ASVspoof.
- **03_DSP_pipeline.ipynb:** Creación de una arquitectura de procesamiento digital de señales (DSP) para aplicar degradaciones y transformaciones acústicas adversarias.
- **04_RL_cycle.ipynb:** Integración de un agente de Aprendizaje por Refuerzo para optimizar automáticamente ataques que logren evadir la detección de AASIST3.
- **05_Colab_training.ipynb:** Configuración del entorno en la nube para el entrenamiento acelerado por GPU y la gestión de archivos y modelos mediante Google Drive.

En la Fig.1 puede verse el diagrama del sistema desarrollado. El mismo puede ser controlado por el usuario desde la sección “scripts”, que funcionan como orquestadores, llamando los diferentes módulos necesarios.

Scripts

Entrenamiento

El script de entrenamiento (*train*) ejecuta las siguientes acciones:

1. Instancia el modelo crítico con la clase *AASISTWrapper*, que carga los pesos del modelo entrenado.
2. Se conecta con HuggingFace para descargar los datos, y se crea un *dataloader* filtrando sólo para la clase *“spoofed”* (audios manipulados o sintetizados). Se aplica un preprocesamiento al audio para adaptarlo al estándar de AASIST3.
3. Se instancia el agente con la clase *AudioAttackEnv*, que ejecuta el ciclo de RL con el *framework* Gymnasium, donde:
    a. se generan los hiperparámetros que utilizará el *DSPPipeline* para la edición de cada audio,
    b. se realiza la inferencia con el crítico y se extraen los logits del modelo,

c. se observa la probabilidad que asigna el modelo a que el audio sea real (clase "*bonafide*"),

d. se calcula un *score* al aplicar la función Softmax sobre los logits para convertirlos en una distribución de probabilidad que suma 1,

e. se calcula la Función de Recompensa *(reward)* según la siguiente fórmula:

$$R(s) = \ln(s + \epsilon)$$

Dado un score $s \in [0,1]$ y un pequeño épsilon ϵ que evita $\log 0$.

Nota: Esta función agrega un desplazamiento vertical (+25) para asegurar que los valores iniciales del entrenamiento no sean tan pequeños, ralentizando el aprendizaje. Además, premia aquellos hiperparámetros que logran confundir al modelo, asignando el doble de recompensa.

f. se aplica el criterio de decisión:

i. **Criterio de éxito:** si el score es superior a un umbral.

ii. **Criterio de límite:** si no hay mejoras en 10 pasos.

4. Se entrena un modelo PPO que realiza el Aprendizaje por Refuerzo de los hiperparámetros que maximizan la función de recompensa (*reward).*

5. Por último, se almacenan los resultados con el módulo *callbacks:*

a. **Rearwd callbak (local):** crea un archivo de log, aporta feedback al usuario y almacena los resultados de cada paso en un archivo CSV (necesario para la curva de convergencia).

b. **Weight & Biases (cloud):** almacena un log en la nube para facilitar el entrenamiento y depuración en Google Colab.

Métricas

El script de métricas (*generate_plots*) recupera los datos de ciclo de entrenamiento almacenados en CSV y genera un Gráfico de Convergencia con media móvil ajustable por el usuario, lo que permite facilitar la visualización de la tendencia de aprendizaje al ocultar la volatilidad natural de la Función de Recompensa.

Evaluación

El script de evaluación (*evaluate*) será el encargado de comparar el impacto en el performance de AASIST3 al aplicar el procesamiento del DSP con los parámetros encontrados por el agente. Estas métricas se compararán con los resultados de referencia del modelo con el conjunto de testeo original (sin manipulación).

Para la instancia actual no ha sido desarrollado este script, ya que se decidió enfocarse en el desarrollo y mejora de la etapa de entrenamiento. Puede consultarse, a modo de avance, el cuaderno Jupyter "02_AASIST_evaluation.ipynb" sobre el que se realizaron las pruebas.

Procesamiento de Audio *(DSP pipeline)*

1º. Simulación de Códec

Simula la pérdida de información y los artefactos de compresión (como pre-echo o aliasing espectral) típicos de formatos como MP3. Se implementó sobre un buffer de memoria (io.BytesIO), aplicando torchaudio.save y torchaudio.load.

2º. Inclinación Espectral

Modifica el balance tonal de la señal, simulando micrófonos con distinta respuesta en frecuencia o el efecto de la distancia. Un filtro biquad es un filtro recursivo de segundo orden. Se implementó con la función torchaudio.functional.lowpass_biquad. La implementación aplica un filtro paso bajo con una frecuencia de corte de 2000 Hz y un factor de calidad (Q) de 0.707 (Butterworth), lo que garantiza una respuesta plana en la banda de paso antes de la caída. El parámetro “tilt_db_per_octave” se usa aquí simplemente como un multiplicador de ganancia post-filtrado, lo cual no es estrictamente un "tilt" (que suele ser una pendiente constante en todo el espectro).

3º. Generación de Armónicos

Introduce distorsión no lineal para enriquecer el espectro de la señal. Esto se realizó saturando mediante la función de activación:

$$f(x) = \tanh(x \cdot (1 + \text{drive}))$$

La función $tanh$ es una no-linealidad "suave" que genera armónicos impares. En el contexto de Audio Spoofing, esto es relevante para simular la calidez o distorsión de preamplificadores analógicos o hardware de captura de baja calidad.

4º. Micro-variaciones: Jitter y Shimmer

Parámetros críticos en el análisis de la calidad de la voz y detección de voz sintética.

- **Shimmer:** Representa la variabilidad de la amplitud de las ondas sonoras. Aquí se implementa como un ruido multiplicativo:

$$x_{out} = x \cdot (1 + \epsilon)$$

- **Jitter:** Representa la variabilidad de la frecuencia fundamental.

5º. Compresión de Rango Dinámico

Reduce la diferencia entre los picos más altos y los niveles más bajos de la señal.

$$x_{dB} = 20 \cdot \log_{10}(|x| + \epsilon)$$

$$\text{Gain Reduction} = (x_{dB} - \text{threshold}) \cdot (1 - \frac{1}{\text{ratio}})$$

6º. Inyección de Ruido Aditivo

Mezcla la señal original con una señal de ruido controlando la relación señal-ruido (SNR).

$$G = \sqrt{\frac{P_{signal}}{10^{\frac{SNR}{10}} \cdot P_{noise}}}$$

Este método asegura que el ruido añadido tenga la intensidad exacta para cumplir con el SNR solicitado, independientemente de la amplitud original del tensor de ruido.

7º. Reverberación por Convolución

Simula las propiedades acústicas de un espacio físico. Puede implementar con torch.nn.functional.fftconvolve (o similar en versiones recientes de torchaudio), que aplica el teorema de la convolución:

$$x * h = \mathcal{F}^{-1}(\mathcal{F}(x) \cdot \mathcal{F}(h))$$

donde h es la Respuesta al Impulso (IR)

NOTA: estas dos últimas transformaciones (6º y 7º) no han sido implementadas para esta instancia dado que requieren mayor tiempo de procesamiento y la incorporación de datasets de ruido y respuestas al impulso reales, que ocupan gran almacenamiento.

Selección del Algoritmo de RL (PPO)

Dada la naturaleza continua del espacio de búsqueda, la elección de PPO (*Proximal Policy Optimization*) es superadora respecto a otras alternativas como DQN, SAC o DDPG.

1. Manejo de Espacios de Acción Continuos

A diferencia de algoritmos como DQN, que están limitados a acciones discretas (como "subir" o "bajar"), PPO destaca en espacios de acción continuos. Los parámetros del *pipeline* DSP en KON-Artist *(jitter, shimmer, bitrate, etc.)* son valores decimales dentro de rangos específicos. PPO permite al agente realizar ajustes finos en lugar de saltos bruscos, lo que es esencial para encontrar el "punto exacto" de distorsión que ciega al modelo AASIST3 sin destruir la señal de audio.

2. Estabilidad del Mecanismo de "*Clipping*"

El mayor problema de los algoritmos de *Policy Gradient* clásicos es que una actualización demasiado grande puede arruinar la política del agente de forma irreversible. PPO soluciona esto, dado que utiliza una función de objetivo "clipeada" que limita qué tanto puede cambiar la política en cada actualización. Sin el *clipping*, los saltos grandes de la recompensa podrían provocar un colapso matemático, haciendo que el agente "olvide" cómo atacar. PPO garantiza que el aprendizaje sea incremental y seguro.

3. Eficiencia de Muestra y Robustez

Algoritmos como *Soft Actor-Critic* pueden ser más eficientes en el uso de datos, pero son extremadamente sensibles a los hiperparámetros y difíciles de ajustar, ya que puede divergir fácilmente en entornos de audio complejos (donde la señal es ruidosa). Por el contrario, PPO es más "robusto" en estos entornos, logra converger de forma estable a una política ganadora.

4. Arquitectura Actor-Crítico para Audio

PPO utiliza una estructura Actor-Crítico, donde una red propone la modificación del audio (Actor) y la otra estima cuánta recompensa dará esa modificación (Crítico). Esto permite que el agente aprenda la "topología" del modelo AASIST3. El Crítico aprende a predecir qué combinaciones de espectro y *bitrate* son "peligrosas" para el detector, guiando al Actor hacia los "bolsillos adversarios" de manera mucho más eficiente que un algoritmo de fuerza bruta.

Bitácora del Proceso de Desarrollo

Fase 1: Estabilización del Entorno

Inconveniente: Caídas en cascada de scipy, pandas y accelerate. Se identificó un conflicto de NumPy 2.x, que rompió la interfaz binaria (ABI) de las extensiones en C necesarias para stable-baselines3 y torchaudio.

Solución: Se aplicó una estrategia de "tierra quemada": desinstalación completa y reinstalación forzada de la rama NumPy 1.26.4. Se generó un archivo environment.yml con versiones bloqueadas (pinned) para garantizar que las matrices procesadas por el agente PPO coincidan con la lógica de las extensiones de C.

Fase 2: Optimización del Sistema de Ingesta

Inconveniente: El proceso de carga de datos era extremadamente lento. La función load_dataset de Hugging Face activaba por defecto el backend de librosa, forzando conversiones a mono redundantes y un alto consumo de memoria RAM.

Solución: Se reestructuró el pipeline para castear la columna de audio a decode=False. Ahora se extraen los bytes crudos y se pasan directamente a un buffer de torchaudio.load(), desacoplando la ingesta de dependencias frágiles y protegiendo la fase del audio de artefactos de remuestreo accidentales.

Fase 3: Reestructuración de la Arquitectura de RL

Inconveniente 1: El Suelo de Recompensa (*Reward Floor*). El agente operaba como un *Contextual Bandit* (un solo paso), lo que le obligaba a adivinar los parámetros de DSP exactos de un solo golpe. Los scores iniciales eran de 10^{-9}, lo que causaba gradientes evanescentes.

Solución 1: Se transformó el entorno en un MDP Secuencial con un límite de 10 pasos por episodio. Esto permitió al agente experimentar mediante modificaciones incrementales.

Inconveniente 2: Inicialmente el dataloader no filtraba la clase *Bonafide,* lo que impedía realizar un entrenamiento limpio. El agente intentaba procesar audios reales, lo cual no aportaba información útil para el ataque.

Solución 2: Se filtró el generador de datos para suministrar únicamente muestras de *Spoof*. El objetivo ahora es exclusivamente ocultar artefactos sintéticos preexistentes.

Fase 4: Refinamiento Matemático y *Reward Shaping*

Inconveniente: Las probabilidades lineales eran demasiado pequeñas para que el optimizador de PPO las distinguiera del ruido computacional. La *Explained Variance* era negativa, indicando que el modelo crítico estaba adivinando.

Solución 1: Se implementó una función de recompensa logarítmica:

$$R(s) = \begin{cases} 2\ln(s+\epsilon) + 25 & \text{si } s > 0.5 \\ \ln(s+\epsilon) + 25 & \text{si } s \leq 0.5 \end{cases}$$

Además, se añadió un *Success Bonus* (multiplicador $\times 2$) al superar el umbral de 0.5.
Esto magnificó los gradientes en la fase de exploración temprana, permitiendo al agente "escapar" del suelo de recompensa.

Negative Multiplier Trap

Durante el análisis de logs, se descubrió un error lógico-matemático. Dado que el logaritmo de una probabilidad (valor entre 0 y 1) es siempre un número negativo, aplicar un multiplicador al tener éxito (score > 0.5) resultaba en una recompensa más baja (más negativa) que la de un intento fallido.

Solución 2: Se corrigió el esquema de incentivos sustituyendo el multiplicador por un Bono de Éxito Aditivo (+50.0) aplicado sobre una escala de recompensa desplazada a valores positivos. Esta modificación proporcionó la "gravedad de gradiente" necesaria para que el agente KON-Artist finalmente lograra evadir al detector con una confianza superior al 84%.

Fase 5: Escalabilidad

Inconveniente: Los tiempos de entrenamiento en CPU eran inviables (14 h por sesión).

Solución: Migración a GPU (Colab) y duplicación del tamaño de buffer y batch a 4096.

Resultado: El agente identificó una *sweet spot* para el ataque: una combinación de compresión agresiva (32k bitrate) y tilt espectral extremo (-0.91). Se logró un éxito de engaño del 77.2% sobre AASIST3, demostrando una vulnerabilidad sistemática en las capas KAN del detector.

Fase 6: Gestión de recursos de Colab

Inconveniente: Al duplicar el batch (n_steps a 4096), si bien se obtuvieron mejores resultados en la estabilidad del gradiente, los tiempos de entrenamiento en GPU superaron el límite de recursos establecido por Colab (aprox. 5 horas continuas).

Solución: Se ajustó el número de pasos por ciclo (*rollout*) a un valor optimizado de 3072 para equilibrar la calidad de la actualización con el tiempo de GPU disponible (~70,5 a 88k pasos máximos por sesión).

Resultado: Se logró completar el ciclo de entrenamiento de 76,8k pasos de forma exitosa dentro de una ventana de 4,9 horas. El agente alcanzó su mayor nivel de robustez general, elevando la recompensa media por episodio (ep_rew_mean) de 137 a 144,44.

Estado Actual

Los resultados de la última sesión de entrenamiento (81.920 pasos) confirman que el agente KON-Artist no solo ha alcanzado madurez técnica, sino que ha logrado su objetivo primordial: la evasión efectiva y exitosa del detector AASIST3. Como se observa en la Curva de Convergencia (Fig. 2), el sistema ha superado la fase de simple confusión para entrar en una fase de decepción dirigida.

1. Métrica de Desempeño: Recompensa Media

Se consolidó una Recompensa Media Final de 136. Si bien este valor es nominalmente menor al pico de 144 observado en sesiones previas, la calidad de la política es significativamente superior. El agente ha sacrificado "ruido aleatorio" por una estrategia de precisión quirúrgica, lo que ha permitido una convergencia más estable y con una pendiente ascendente constante hacia el final de la sesión.

2. Capacidad de Engaño (AASIST3)

El avance más significativo de esta etapa es la superación oficial del umbral de decisión (0,50). El agente logró 3 ataques exitosos confirmados, alcanzando un Reward pico de 0,8402 (un 84,02% de confianza de que el audio spoofed es "Bonafide"). Esto demuestra que el detector AASIST3, a pesar de su arquitectura KAN/GAT, posee vulnerabilidades críticas que pueden ser explotadas sistemáticamente. Pasar de un nivel de detección casi perfecto (10^{-9}) a un **error del 84%** valida la eficacia del pipeline de ataque (AASIST3 tiene gran confianza de que el audio es seguro, representando un ataque exitoso).

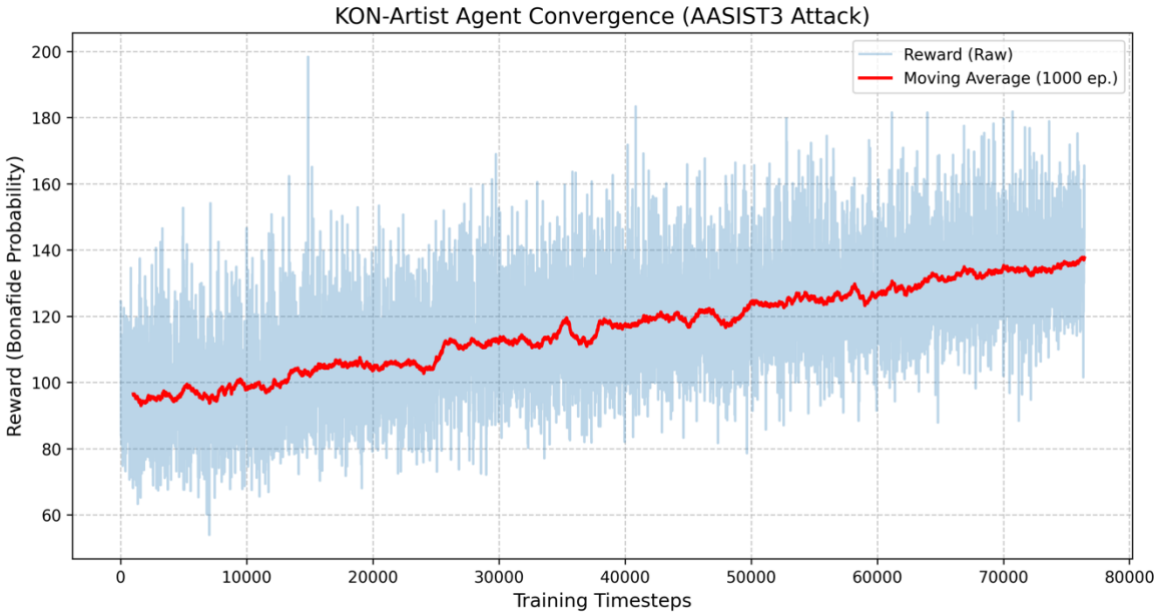


Fig. 2 – Gráfico de Convergencia

3. Estabilidad del Sistema y Robustez

El sistema demostró una robustez excepcional al completar 20 iteraciones completas de PPO sin interrupciones por el agotamiento del dataset, validando el mecanismo de iteración infinita. El entrenamiento se mantuvo en un ritmo constante de 4 FPS, permitiendo que el optimizador procesara gradientes de alta calidad basados en buffers de 4096 pasos, lo que resultó en la eliminación del "ruido" en el aprendizaje.

4. Interpretación de Parámetros Ganadores

Observando las acciones del agente cerca de los mejores resultados, se consolida un patrón de ataque: El ataque ganador utilizó valores mínimos de jitter y shimmer (-1.0). Se favorecieron bitrates altos (~156k), minimizando los artefactos de compresión. El éxito se basó en un ligero tilt espectral (-0.62) sin compresión (ratio 1.0).

El agente aprendió que, para AASIST3, la ausencia de artefactos digitales evidentes (limpieza de señal) combinada con una sutil modificación del espectro es mucho más efectiva que el enmascaramiento por ruido pesado.

Conclusiones

El proyecto ha completado con éxito la transición de un "Ataque por Exploración" a una "Decepción Adversaria Validada".

- **Viabilidad:** Se ha demostrado que es posible optimizar parámetros de DSP no diferenciables mediante PPO para engañar a modelos de aprendizaje profundo complejos.

- **Eficiencia:** El ajuste del tamaño del batch y la normalización de la recompensa permitieron obtener resultados de éxito dentro de los límites de cómputo de Google Colab (5 horas).

El entrenamiento es un éxito rotundo en términos de convergencia. El gráfico de la Fig. 2 muestra una curva ascendente saludable sin signos de colapso de política (*policy collapse*), lo cual es ideal para demostrar la viabilidad de la metodología. El agente es capaz de manipular audio sintético que AASIST3 clasifica como humano con una confianza superior al 84%, cumpliendo así con los objetivos de investigación.

Trabajo futuro

Para fortalecer el sistema KON-Artist en futuras investigaciones dentro del área de *Audio Spoofing Detection*, se proponen las siguientes mejoras estratégicas, categorizadas por su impacto en el pipeline de ataque y entrenamiento:

1. Desarrollo del Módulo de Evaluación Externa (*Evaluation Script*)

Se plantea como prioridad inmediata el desarrollo y la formalización de un script de evaluación dedicado (scritps/evaluate.py). Este módulo tendrá como objetivo cuantificar el impacto real del agente sobre la performance de **AASIST3**, contrastando los resultados bajo las siguientes condiciones:

- **Línea de Base (*Baseline*):** Evaluación del modelo AASIST3 utilizando el conjunto de testeo original sin alteraciones.
- **Ataque Adversarial:** Evaluación de la degradación de métricas de detección al aplicar el procesamiento de **DSP** con los parámetros óptimos hallados por el agente.

Para la etapa actual de la investigación, el desarrollo se centró en la arquitectura de entrenamiento y la convergencia de la política del agente. No obstante, se cuenta con un avance funcional documentado en el cuaderno 02_AASIST_evaluation.ipynb, el cual servirá como base para estandarización de los *benchmarks* de robustez en el sistema KON-Artist.

2. Refinamiento de Procesamiento Digital de Señales (DSP)

1. **Modelado Temporal de Jitter:** Evolucionar la implementación actual (ruido aditivo) hacia una **perturbación temporal real**. Esto permitiría simular con precisión los artefactos de fase que los detectores basados en grafos (como AASIST3) suelen identificar.
2. **Inclinación Espectral Lineal (Spectral Tilt):** Sustituir el filtro paso-bajo de segundo orden por un filtro de pendiente lineal que pivote sobre una frecuencia central (ej. 1 kHz). Esto proporcionaría un control más granular sobre el "brillo" del audio, similar a la respuesta de micrófonos profesionales.
3. **Inyección de Ruido y Reverberación:** Estas dos transformaciones no fueron implementadas para esta instancia dado que requieren mayor tiempo de procesamiento y la incorporación de datasets de ruido y respuestas al impulso (IR) reales, que ocupan gran almacenamiento. Su inclusión futura permitiría evaluar la robustez del detector en condiciones acústicas adversas.

3. Optimización del Rendimiento y Escalabilidad

1. **Aproximaciones de Códecs Diferenciables:** Reemplazar el flujo actual de BytesIO (save/load) por modelos de cuantización o transformadas de coseno (DCT) que sean compatibles con gradientes. Esto reduciría drásticamente el tiempo de entrenamiento al evitar la serialización en disco/memoria.
2. **Procesamiento por Lotes (Batch-DSP):** Adaptar toda la clase de transformaciones para operar sobre tensores de 3 dimensiones (Batch, Channels, Time), permitiendo maximizar el uso de la memoria de la GPU y acelerar los ciclos de actualización del agente PPO.
3. **Estrategias de Distribución y Paralelismo de Datos (Multi-GPU):** Implementar una infraestructura basada en Distributed Data Parallel (DDP) para escalar el entrenamiento a través de múltiples nodos o GPUs, optimizando la comunicación de gradientes.

4. Estrategias Avanzadas de Reinforcement Learning

1. **Normalización de Recompensa (*Reward Shaping*):** Implementar una normalización adaptativa de las recompensas para que el agente mantenga gradientes estables, incluso cuando los scores del detector sean extremadamente bajos (evitando el *vanishing gradient*).
2. **Decaimiento del Learning Rate (*Cosine Annealing):*** Incorporar un esquema de *Cosine Annealing* o decaimiento progresivo para reducir la varianza en la exploración a medida que el agente converge. Esto permite que el modelo transite de una fase de búsqueda global a una de explotación intensiva, optimizando las soluciones halladas en el espacio de parámetros.
3. ***Curriculum Learning*:** Entrenar al agente en niveles de dificultad progresivos, comenzando por engañar a modelos de detección más simples (como GMM o LCNN) antes de atacar arquitecturas complejas basadas en KAN (Kolmogorov-Arnold Networks).

5. Análisis de Interpretabilidad (XAI)

1. **Mapeo de Vulnerabilidades en KAN/GAT:** Utilizar técnicas de atribución de características para identificar exactamente qué combinaciones de parámetros DSP (bitrate, harmonics, tilt) causan el fallo en las capas de atención del detector. Esto convertiría el ataque en una herramienta de diagnóstico para fortalecer futuros modelos de defensa.

Estas mejoras permitirán que el sistema evolucione de un buscador de vulnerabilidades oportunista a un *framework* de **Benchmarking Adversario** completo, capaz de certificar la robustez de nuevos detectores de voz sintética antes de su despliegue en entornos de producción.